Saturday, November 22, 2025 2:32 PM

State Management on Stateless HTTP

What Does “Stateless HTTP” Mean?

HTTP, the protocol used by browsers to communicate with servers, is **stateless**.
This means:
Each request from the browser is treated as a completely new request.
The server does not remember anything about previous requests.

Why Is State Management Needed?

Modern web applications need to **remember user information** across multiple requests.
Examples:

- Remembering a logged-in user
- Keeping items in a shopping cart
- Remembering preferences like theme or language
- Tracking page navigation

Since HTTP forgets everything after each request, developers must create techniques to **store and manage information (state)**.
This process is called:

State Management

It means **preserving data across multiple HTTP requests** so the user experience feels continuous and connected.

How State Management Helps

State management allows applications to:

- Identify users from request to request
- Maintain login sessions
- Store temporary or persistent values
- Improve performance by avoiding repeated computation
- Personalize pages based on stored data

Without state management:

- Every click would require the user to “start over”
- No login session would work
- Cart items would be lost instantly
- Websites would feel broken and disconnected

Where Can State Be Stored?

State can be stored in:

- The **client side** (browser)
- The **server side** (server memory or database)

Server-Side State Management

When a user browses a web application, each request they send is separate.
HTTP does **not remember** anything — it is *stateless*.

So if a user moves from one page to another, the server forgets who they are and what they were doing.
Server-side state management solves this problem.
It means **the state (data) is stored on the server**, not on the user's browser.

Why Server-Side State Management?

- The server is more **secure** than the client browser.
- Can store **large amounts of data**.
- Ideal for **user-specific, sensitive, and temporary information**.
- Prevents tampering because the user cannot directly see or change the data.

1. Session State

Session State is a technique where the **server stores user-specific data** for a short period while the user is active on the website.

Why Session State?

Session State helps the server:

- remember **who the user is**
- remember **what they were doing**
- store **temporary data** safely on the server
- track a user **across multiple pages**

SET SESSION DATA

```
public IActionResult SetSession()
{
    HttpContext.Session.SetString("UserName", "Aashu");
    HttpContext.Session.SetInt32("UserAge", 21);

    return Content("Session values saved.");
}
```

READ SESSION DATA

```
public IActionResult ReadSession()
{
    string name = HttpContext.Session.GetString("UserName");
    int? age = HttpContext.Session.GetInt32("UserAge");

    return Content("Name: " + name + ", Age: " + age);
}
```

Characteristics

- Data is stored on the **server**, not on the browser.
- Each user gets a **unique session ID**.
- Session lasts until:
    - user closes browser
    - session timeout completes
    - server clears it manually
- Good for storing sensitive or larger data (because it's on the server).

Simple Example

You open a shopping website.

Step 1: You add a shoe to your cart

When you click “Add to Cart”, the server saves:

- product ID: 101
- size: 42
- price: 2000

This information is stored **inside your session** on the server.

Step 2: You open a different page

Maybe “Men’s Jackets” page.
Even though it’s a totally different request, the server uses your **session ID** to link back to your session locker and remembers:

- you already added a shoe to cart

Step 3: You add a jacket

Now the server updates your session locker:

- product ID: 101 (shoe)
- product ID: 220 (jacket)

Step 4: You check out

The cart information is still there because your session was active.

Step 5: You close the browser

Your session ends after some time, and the server clears the locker.

When to Use Session State

Use it when:

- you need to keep user-specific data across pages
- the data is sensitive (server storage is safer)
- the data size is big (cookies or query strings won't fit)

Examples:

- Shopping cart
- Logged-in user information
- Saving quiz progress
- Temporary user preferences

2. TempData

ViewData

- Used to pass small pieces of data **from Controller → View**.
- Works **only for the current request**.
- Data is stored as **key–value pairs**.
- Requires **type casting** because it stores data as object.

Use when: you need to send temporary UI data to the view (titles, messages, etc.).
ViewData["Title"] = "Student List";
ViewData["TotalStudents"] = 30;
Keys: strings
Values: objects
Must cast while using

TempData

- TempData is also **key–value storage** like ViewData.
- But **it survives one extra request**.
- Uses **session** internally.
- Data is removed automatically after it is read once (one-time message system).

Use when:

You need to pass data **from one action to another** using **Redirects**.
Example uses: success messages, warnings, form submission results.

How TempData Behaves

- Action 1 sets TempData
- Redirects to Action 2
- Action 2 receives the data

ViewData cannot do this (dies in the same request).
Session can do this, but it stays for much longer and must be manually cleared → too heavy for simple messages.
So TempData is a perfect **middle option**.

Advantages of TempData

i. Works across redirects

- This is its biggest advantage.
- Supports *Post → Redirect → Get* (PRG) flow.
- Ideal for login messages, form submissions, success/failure notices.

ii. Automatically removed after reading

- Keeps memory clean.
- No chance of stale values staying for long.

iii. Uses Session internally but behaves lighter

- It doesn't remain for the whole session.
- Works only for next request.

iv. Simple to use

- No configuration required.
- Stores small temporary information easily.

v. Better than ViewData when you need more than one request

- ViewData = current request only
- TempData = survives one redirect
- Session = survives entire session → too long
  TempData sits right in the sweet spot.

3. HttpContext

HttpContext represents **all information about the current HTTP request and response** in an ASP.NET Core application.

What it contains

- **Request** details (URL, headers, form data, query strings)
- **Response** details (status code, headers)
- **Session** data
- **User** identity (logged-in user info)
- **Connection** info (client IP, protocol)
- **Cookies**
- **Items** (temporary data for this request)

Why it is important

- It allows the application to **understand who is making the request** and **what they are asking for**.
- Helps manage session, cookies, authentication, and request data.

4. Cache

Cache in ASP.NET Core is a **server-side state management technique** used to store frequently used data temporarily for faster performance.

Why caching is used

1. **Improves performance** — reduces time to fetch data.
2. **Reduces database load** — data is fetched once and reused.
3. **Speeds up page loading** — improves user experience.
4. **Efficient for costly operations** — like reports or calculations.

Characteristics

- Data is stored **in server memory**.
- Cached data is shared among all users.
- It expires after a set time or when memory is needed.
- Suitable for data that **does not change frequently**.

What type of data is cached

- Product lists
- Categories
- Dashboard statistics
- Frequently accessed information
- Lookup tables (countries, districts, etc.)

Benefits of Cache

- **Fast access:** reduces repeated computation or DB queries.
- **Scalable:** makes the application handle more users easily.
- **Cost-effective:** saves processing and infrastructure cost.
- **Better user experience:** pages load smoother and quicker.

Client-Side State Management

Client-side state management means **saving small pieces of data on the user's browser**, not on the server.
This reduces server load and helps remember user activities between requests.

Why client-side?

- ASP.NET Core apps run on HTTP, which is **stateless**
- Browser forgets everything after each request
- Client-side techniques help **remember small information** such as:
    - Username
    - User preferences
    - Selected theme
    - Items in a small cart
    - Form inputs

Common Client-Side Strategies

1. **Cookies**
2. **Query Strings**

3. **Hidden Fields**

1. Cookies

Cookies are **small text files** stored in the user's browser.
Used to remember information between multiple page visits.

Key Features

- Stored on client machine (browser)
- Very small (around 4KB each)
- Sent with **every request** to the server
- Can have an **expiry time** (short-term or long-term)
- Useful for:
    - Remembering logged-in user
    - Preferences (theme, language)
    - Tracking visited pages
    - Simple personalization

Advantages of Cookies

- Do not overload the server (stored on client)
- Persistent (can stay for days/months)
- Automatic (browser sends them on every request)
- Simple to use

Disadvantages

- Size limit (4KB)
- Users can delete them
- Not good for sensitive information
- Visible to the user (not fully secure)

Cookie Format

A cookie is stored like:

UserTheme = Dark
UserId = 101
Language = English
Each cookie has **Key = Value**.

Syntax to Create a Cookie (ASP.NET Core)

Setting With Expiry

```
Response.Cookies.Append("UserName", "Aashu",
  new CookieOptions
  {
    Expires = DateTime.Now.AddMinutes(10)
  });
```

Reading a Cookie

```
string user = Request.Cookies["UserName"];
```

If cookie doesn't exist

The variable becomes **null**, so often checked like:

```
if (user != null)
{
   // use the value
}
```

Deleting a Cookie

```
Response.Cookies.Delete("UserName");
```

2. Query String

What it is:
A query string is a small piece of data added to the **URL** when sending information from one page to another.
It always appears **after a "?"** in the URL.
Looks like:
https://example.com/profile?userid=10&name=aashu
How it works:

- Browser sends data to the server through the URL.
- Data is visible to everyone because it is inside the URL.

When to use:

- When data is **not sensitive** (like page number, search term, sorting type).
- When you want **bookmarkable** or **shareable** links.
- For simple, small pieces of data.

How you create:
You usually add key-value pairs to a URL:
?key=value

3. Hidden Field

A hidden field is a small storage space inside an HTML form that keeps data **secretly** (not visible to the user) but still sends it to the server when the form is submitted.

Sometimes you want to remember something between requests (like user ID, product ID, token), but you don't want the user to see or edit it directly. Hidden fields quietly carry that data for you.

How a Hidden Field Looks

Hidden fields are created using an HTML input element with type set to **hidden**.
Example :

```
<form action="/Submit" method="post">
   <!-- Hidden data stored secretly -->
   <input type="hidden" name="userid" value="10">
<!-- Visible input -->
   <input type="text" name="message" placeholder="Enter message">
<button type="submit">Send</button>
</form>
```

Here,

- The user can see only the text box and button.
- The hidden field (userid = 10) is not visible, but when the form is submitted, the server receives both:

- message = whatever user typed
- userid = 10

When to Use Hidden Fields

- To pass **ID values** between pages (user ID, product ID).
- To keep **temporary data** during form submissions.
- To hold values that the server needs but the user should not edit directly.
- To maintain data **without using session or cookies**.

Important Note

Hidden fields are not truly safe. Users can still change them using "Inspect Element," so never use them for sensitive or secure values.

Common Client-side Web Technologies

When a user opens your website, two sides start working:
the server side (ASP.NET Core in our case) and **the client side** (the browser).
Client-side technologies run *inside the user's browser*. They make the website **faster, smoother, and more interactive** without needing to contact the server again for every small action.

1. jQuery

Introduction:

jQuery is a fast, small, and feature-rich JavaScript library. It simplifies tasks like HTML document traversal, event handling, animations, and AJAX. It is widely used for client-side development in web applications, including ASP.NET Core projects.

Features of jQuery:

- **Simplified DOM Manipulation:** Easily select, modify, and traverse HTML elements.
- **Event Handling:** Handle user interactions (click, hover, keypress) with minimal code.
- **AJAX Support:** Simplifies asynchronous requests to fetch data from the server without reloading the page.
- **Cross-Browser Compatibility:** Works consistently across all major browsers.
- **Animations and Effects:** Add visual effects like fading, sliding, hiding/showing elements.
- **Plugins:** Extend functionality using thousands of available plugins.

Why use jQuery:
It reduces the amount of JavaScript code you write, improves readability, and handles browser differences automatically. Ideal for form validation, dynamic content, and interactive UI features.

jQuery Syntax

Every jQuery code starts with the **$ symbol**.
General syntax:

```
$(selector).action()
```

- **$** → jQuery object

- **selector** → selects HTML element (e.g. "#btn", ".class", "p")
- **action()** → what you want to do (e.g. hide, show, validate, click)

Example:

```
$("#btn").click(function() {
   $("#msg").text("Button clicked");
});
```

jQuery Selectors

Selectors are used to **select HTML elements** so we can perform actions on them. They are very similar to CSS selectors.

Types of Selectors:

ID Selector: Selects element by its ID.

```
$("#name") // selects element with id="name"
```

Class Selector: Selects elements by their class.

```
$(".inputField") // selects all elements with class="inputField"
```

Element Selector: Selects elements by their tag name.

```
$("input") // selects all input elements
```

Attribute Selector: Selects elements by attributes.

```
$("input[type='text']") // selects all text input fields
```

Combination Selectors: Combine selectors for more precise selection.

```
$("form#studentForm input") // selects all input fields inside the form with id="studentForm"
```

jQuery Actions

Actions are things we **do to the selected elements**, like changing values, hiding/showing, or reacting to events.

Common Actions:

.val() – Get or set the value of an input.

```
var name = $("#name").val(); // get value
$("#name").val("John");     // set value
```

.html() / .text() – Get or set the content inside an element.

```
$("#error").html("Please enter your name");
```

.hide() / .show() / .toggle() – Hide or show elements.

```
$("#message").hide();
$("#message").show();
```

.css() – Change CSS styles dynamically.

```
$("#name").css("border", "1px solid red");
```

Event Actions – React to user events like click, hover, submit.

```
$("#submitBtn").click(function() {
   alert("Button clicked!");
});
```

Scenario

We want a form to take student data:

- Student Name (non-empty)
- Email (must be a valid email)

- Phone Number (exactly 10 digits)
- Number of Books Borrowed (positive number)

We will **validate this using jQuery** before submitting the form.

```
<html>
<head>
  <title>Student Form Validation</title>

  <!-- jQuery library -->
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>

  $(document).ready(function() {
    $("#submitBtn").click(function() {
      var name = $("#studentName").val();
      var email = $("#studentEmail").val();
      var phone = $("#studentPhone").val();
      var books = $("#numBooks").val();
      var errorMsg = "";
// Name validation
      if(name == "") {
        errorMsg += "Please enter Student Name.<br>";
      }
// Email validation )
      var emailPattern = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,6}$/;
      if(email == "" || !emailPattern.test(email)) {
        errorMsg += "Please enter a valid Email.<br>";
      }
// Phone validation
      var phonePattern = /^[0-9]{10}$/;
      if(!phonePattern.test(phone)) {
        errorMsg += "Phone number must be 10 digits.<br>";
      }
// Number of books validation
      if(books == "" || books <= 0) {
        errorMsg += "Number of books must be a positive number.<br>";
      }
// Show errors if any
      if(errorMsg != "") {
        $("#errorDisplay").html(errorMsg).slideDown();
        return false; // Stop form submission
      }
    });
  });
  </script>
</head>
<body>

    <form method="POST" action="">
      <label id="errorDisplay" style="color:red;"></label><br>
<label>Student Name:</label>
      <input type="text" id="studentName"><br><br>
<label>Email:</label>
      <input type="text" id="studentEmail"><br><br>
<label>Phone Number:</label>
      <input type="text" id="studentPhone"><br><br>
```

```
<label>Number of Books Borrowed:</label>
        <input type="number" id="numBooks"><br><br>
<button type="submit" id="submitBtn">Submit</button>
      </form>
   </fieldset>
</body>
</html>
```

Explanation

1. **jQuery Setup:**
    - <script src="..."></script> loads the jQuery library.
    - $(document).ready() ensures the DOM is fully loaded before running code.
2. **Event Handling:**
    - $("#subm").click(function(){ ... }) triggers when the user clicks the login button.
3. **Selectors:**
    - $("#studentName"), $("#studentEmail"), $("#studentPhone"), $("#numBooks") select the input fields.
4. **Validation Logic:**
    - Checks for empty student name.
    - Uses a regex pattern for email validation.
    - Ensures phone number is exactly 10 digits using regex.
    - Checks number of books is positive.
5. **Error Handling:**
    - All validation messages are collected in errorMsg.
    - If any error exists, it is displayed in #errorDisplay using slideDown().
    - Stops form submission with return false.
6. **User Experience:**
    - Multiple errors show together.
    - Form submission only happens if all inputs are valid.

Single Page Application (SPA)

Introduction:

- A SPA is a web application that loads a **single HTML page** and dynamically updates content as the user interacts with the app, without refreshing the entire page.
- It provides a **smooth, app-like experience** for users.

Architecture:

1. **Client-side (Front-end):**
    - Uses frameworks like **Angular, React, or Vue.js** to manage the UI and render views dynamically.
    - Makes **AJAX or API calls** to the server to fetch or send data in JSON format.
2. **Server-side (Back-end):**
    - Provides **APIs (Web API controllers)** to handle requests from the client.
    - Handles database operations, authentication, and business logic.

Features of SPA:

- Fast and responsive because only part of the page updates.
- Smooth user experience like mobile apps.
- Reduced server load; server mainly serves APIs.
- Easier to integrate with **RESTful APIs** and backend services.
- Works well with **ASP.NET Core** by using API controllers for data handling.

Example (in words):

- When you use Gmail, you can click inbox, sent, or compose mail without the page reloading. Only the relevant section updates. This is a SPA in action.

Angular (for SPA with ASP.NET Core)

Introduction:

- Angular is a **front-end framework** for building **Single Page Applications (SPA)**.
- It helps create **dynamic, client-side applications** that interact with ASP.NET Core back-end via APIs.
- The front-end (Angular) handles views, routing, and user interactions, while ASP.NET Core provides data through **Web API controllers**.

Advantages of Angular:

- **Two-way data binding:** Changes in UI and data are automatically synced.
- **Component-based architecture:** Makes code reusable and organized.
- **Rich ecosystem:** Has built-in features for routing, forms, HTTP requests, and more.
- **SPA support:** Works efficiently with APIs for dynamic page updates without full reload.
- **TypeScript support:** Strong typing helps catch errors early and makes code more maintainable.

Disadvantages of Angular:

- **Steep learning curve:** Has many concepts like modules, decorators, and RxJS.
- **Performance overhead:** Large apps may feel slower if not optimized.
- **Verbose code:** Writing components and services can be more detailed than other frameworks.

Example :

- In an online library app: Angular manages the **student form, list of borrowed books, and search feature** dynamically, while ASP.NET Core APIs provide the data for students, books, and transactions.

React (for SPA with ASP.NET Core)

Introduction:

- React is a **JavaScript library** used to build **Single Page Applications (SPA)**.
- Unlike Angular, React focuses on the **view layer** only, so we often use it with **ASP.NET Core APIs** to fetch and update data.
- React uses **components** to build UI, and **state and props** to manage data dynamically.

Advantages of React:

- **Virtual DOM:** Updates only parts of the page that change, making it fast.
- **Reusable components:** Easier to manage large applications.
- **Simpler learning curve** than Angular for basic apps.
- **Flexibility:** Can be used with any back-end (ASP.NET Core, Node.js, etc.).
- **Strong community support** and lots of libraries for forms, routing, etc.

Disadvantages of React:

- **Not a full framework:** Requires additional libraries for routing, state management, etc.
- **JSX syntax:** Can be confusing at first for beginners.
- **Frequent updates:** New versions can require code adjustments.

- **Component:** Small UI piece (StudentForm, ErrorMessage).
- **State:** Keeps track of dynamic data (input values, errors).
- **Props:** Passes data to child components.
- **JSX:** HTML-like syntax to render UI.
- **Event Handling:** Handles user actions like typing and clicking.
- **Controlled Components:** Input value linked to state, helps with validation.

Using React Form Validation Example (Library Management System)

Scenario:

We have a form for **student registration** in a library system. The form collects:

- Student Name (required, cannot be empty)
- Email (must be valid)
- Phone Number (exactly 10 digits)
- Number of Books Borrowed (positive number)

How React handles validation:

1. **Controlled Components:**
   - Each input field is linked to the **state** of the component.
   - Whenever the user types, the state updates automatically.
2. **Real-time Validation:**
   - React checks the input values **as the user types** or when the form is submitted.
   - Example checks:
     - Name is not empty → if empty, show "Please enter your name."
     - Email matches a valid email pattern → if invalid, show "Enter a valid email address."
     - Phone number length is 10 digits → if not, show "Phone number must be 10 digits."
     - Books borrowed is positive → if not, show "Enter a valid number of books."
3. **Error Messages:**
   - Errors are stored in the component state and displayed **next to the field** dynamically.
   - When the user fixes the input, the error disappears immediately.
4. **Form Submission:**
   - React checks if **all fields are valid** before submitting.
   - If valid, data is sent to the **ASP.NET Core backend** via an API call.
   - If invalid, the submission is blocked and relevant error messages are displayed.

- A student enters their details in the form.
- React instantly validates each field:
  - Name typed correctly?
  - Email valid? N0 → Error message appears
  - Phone number 10 digits?
  - Number of books positive?
- The student fixes the email → the error disappears.
- Once all fields are valid, React sends the data to the backend, and the student is registered successfully.

Benefits:

- No page reload is needed.
- User gets **immediate feedback**.
- Reduces invalid data being sent to the server.
- Works smoothly with **ASP.NET Core APIs** for database storage.

comparison between **React** and **Angular** :

Feature	**React**	**Angular**
Type	Library for building UI	Full-fledged framework
Language	JavaScript (JSX)	TypeScript
Data Binding	One-way binding (state → UI)	Two-way binding (UI ↔ state)
Learning Curve	Easier to learn	Steeper, more concepts to learn
Use Case	Flexible, can integrate with any backend	Best for large-scale enterprise apps

Securing in ASP.NET Core Application

Authentication

Authentication is simply the process of **checking who is making a request**. Think of it like showing your ID at the entrance of a building. Once the system knows who you are, it can decide whether you have the right to do certain actions—this is called **authorization**.

Example:

- You log into a website. Authentication checks your username and password.
- After logging in, the system checks whether you are an admin or a regular user to decide what pages you can access.

ASP.NET Core Identity

ASP.NET Core Identity is like a **ready-made login and user management system** for your applications. It helps you manage:

- **Users** – Who can log in
- **Passwords** – Safely stored and hashed
- **Roles** – Admin, manager, user, etc.
- **Claims** – Additional user information like email, age, or subscription level
- **Tokens** – For password resets or email confirmation
- **Profile data** – Like name, address, and preferences

Users can either:

1. **Create an account** directly on your app (username/password stored in Identity).
2. **Use an external login** such as Google, Facebook, Microsoft, or Twitter.

Identity usually stores this information in a **SQL Server database**, but you can also use other storage options like **Azure Table Storage**.

Components of Identity

- **Identity (Microsoft.AspNet.Identity)** – The main library for handling authentication
- **User Store <T> (Microsoft.AspNet.Identity.EntityFramework)** – Manages how user data is stored
- **Sign In Manager** – Handles login and logout operations
- **User** – Represents the user account
- **IdentityDB <T>** – Database context used to store users and roles
- **Identity Middleware** – Connects Identity to your app's request pipeline

Steps to Add Authentication to Apps and Configure Identity Services

Here's a step-by-step story of **adding Identity to your app**:

Step 1: Install Required Packages

From **NuGet**, install:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- Microsoft.AspNetCore.Identity.EntityFrameworkCore

Explanation:

These packages provide the database connection and Identity functionality. Think of them as **the tools needed to build your login system**.

Step 2: Create AppDbContext

Create a class **AppDbContext** in your **Models** folder, inheriting from IdentityDbContext:

```
public class AppDbContext : IdentityDbContext
{
   public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
}
```

Explanation:

- IdentityDbContext automatically creates tables for users, roles, and claims.
- DbContextOptions tells the context **how to connect to your database**.

Step 3: Configure Services in Startup.cs

Inside **ConfigureServices** method, add:

```
services.AddDbContext<AppDbContext>(options =>
   options.UseSqlServer("YourConnectionString"));
services.AddIdentity<IdentityUser, IdentityRole>()
      .AddEntityFrameworkStores<AppDbContext>();
```

Explanation:

- AddDbContext → Connects your app to the SQL Server database.
- AddIdentity → Registers Identity and tells it to use AppDbContext for storing users and roles.
- IdentityUser → Represents a user, IdentityRole → Represents a role.

Step 4: Enable Authentication Middleware

Inside **Configure** method, add:

```
app.UseAuthentication();
app.UseAuthorization();
```

Explanation:

- UseAuthentication() → Checks if a user is logged in for every request.
- UseAuthorization() → Checks if the logged-in user has permission to access a page or action.

Step 5: Create Migration

In **Package Manager Console**, type:

```
Add-Migration AddingIdentity
```

Explanation:

This generates the **database tables** for Identity (users, roles, claims). Think of it as **creating empty folders for storing user information**.

Step 6: Apply Migration to Database

Run:

```
Update-Database
```

Explanation:

Applies the migration to the database, **creating all the tables** so your app can start storing user data.

Authorization

Authorization decides **what a user is allowed to do** after the system knows who they are.

Roles

Roles are like **job titles or access levels**. You assign permissions based on a role, not individual users.
example:

- In **GitHub**, only users in the **Admin role** can manage repository settings.
- A repository may allow access to users in **Developer** or **Tester** roles for editing code.

Claims

Claims are **attributes about a user** (name-value pairs), not their roles. Access is granted based on the **value of a claim**.
example:

- In **Netflix**, users with a **"subscriptionType = Premium"** claim can watch 4K content.
- Users with "subscriptionType = Basic" cannot access those movies.

Policies

Policies are **custom rules** that can combine roles, claims, or other conditions. All policies applied must pass for access.
example:

- In **HR software**, employees can view their own **pay slips** (EmployeeOnly policy).
- Only **HR department employees** can update salaries (EmployeeOnly + HumanResources policies).

    - **Roles** → Access based on user role (Admin, HR, Finance)
    - **Claims** → Access based on attributes about the user (age, membership, status)
    - **Policies** → Access based on custom rules that may combine roles, claims, or other conditions

Difference Between Roles, Claims, and Policies

Feature	Roles	Claims	Policies
What it checks	User's role (job title)	User's attributes (name-value pairs)	Custom conditions combining roles/claims
Example	Admin, Developer, Tester	SubscriptionType = Premium	EmployeeOnly + HumanResources
Scope	Broad (entire controller/action)	Fine-grained (specific attributes)	Flexible, can combine multiple rules
Usage	Simple role-based access	Attribute-based access	Complex business logic and rules

Common Vulnerabilities

Web applications can have weaknesses that attackers exploit to steal data, manipulate content, or take control of user sessions. Understanding these vulnerabilities helps developers **write safer, more secure apps**.

1. Cross-Site Scripting (XSS) Attacks

What it is:
Cross-Site Scripting (XSS) happens when an attacker injects **malicious scripts (usually JavaScript)** into a website. When other users visit the page, the script runs in their browser without their knowledge.
example:

- Imagine a **social media site** where users can post comments.
- If the site does not check the input, an attacker can post a comment like <script>stealCookies() </script>.

- When other users open that comment, the script could **steal their login cookies**, **change what they see on the page**, or **redirect them to a phishing site**.

How it happens:
XSS usually occurs when an application **takes user input and displays it on a page without validating or encoding it**.

Prevention Steps

1. **Never put untrusted data directly into HTML**. Always validate first.
2. **HTML encoding:** If you need to display untrusted data inside an HTML element, encode it so scripts can't run.
3. **HTML attribute encoding:** If putting untrusted data in attributes (like src or title), encode it.
4. **JavaScript safety:** If putting untrusted data into JS, store it in an HTML element and retrieve it safely at runtime.
5. **URL encoding:** If putting untrusted data into query strings, make sure it's properly URL encoded.

Story analogy:
Think of XSS like a **bad message slipped into a company notice board**. Anyone reading it could get tricked, but if you **check and sanitize every message before posting**, you can prevent harm.

2. SQL Injection (SQLI) Attacks

SQL Injection happens when an attacker tricks your application into running *their own SQL code* instead of the intended query.
It usually occurs when you directly insert user input into an SQL query **without checking or protecting it**.
Think of it like this:
You ask a user, "What's your name?" but instead of a name, they give you a command — and your system listens and executes it.
This can expose sensitive data like passwords, emails, user accounts, or even delete the entire database.

How SQL Injection Happens

Imagine a login form:

- You type your username → the app inserts it into an SQL query
- But if an attacker types SQL code, the app also inserts *that code*
- The database executes it as a valid command

That's the whole problem:
User input becomes part of the SQL command.

Why SQL Injection Happens

It happens when:

- The website directly uses user input inside SQL
- No validation is done
- No parameterized queries are used

Example 1: SQL Injection With 1=1 (Always True Condition)

Suppose the website expects a **User ID**:

User ID: 10
The website builds this SQL:

```
SELECT * FROM Users WHERE UserId = 10;
```

Now an attacker enters:

10 OR 1=1
The final SQL becomes:

SELECT * FROM Users WHERE UserId = 10 OR 1=1;
Why dangerous?

- 1=1 is always TRUE
- So the query returns **all users**
- Attacker sees every record → names, emails, passwords, etc.

Example 2: SQL Injection With ' OR '=' (Also Always True)

Attacker enters this in username and password fields:

Username: ' OR '='
Password: ' OR '='
The backend builds this SQL:

SELECT * FROM Users
WHERE Name = '' OR '='
  AND Pass = '' OR '=';
Everything becomes TRUE, so:

- All users are returned
- Attacker may bypass login
- They get into the system without knowing any real password

1. Program Showing the Possibility of SQL Injection

Unsafe Example (Vulnerable Code)

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
# create table
cur.execute("CREATE TABLE users(username TEXT, password TEXT)")
cur.execute("INSERT INTO users VALUES('aashu', '1234')")
# user input (simulating an attacker)
username = input("Enter username: ")
password = input("Enter password: ")
# UNSAFE: building SQL directly from user input
query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password +
"';"
print("Running query:", query)
cur.execute(query)
result = cur.fetchall()
if result:
   print("Login successful")
else:
   print("Login failed")
```

Explanation

- The SQL query is built by **joining user input directly into the string**.
- If an attacker enters:

username: ' OR '1'='1
password: anything
The query becomes:

SELECT * FROM users WHERE username='' OR '1'='1' AND password='anything';

'1'='1' is always true, so the database returns a row and the attacker logs in **without knowing any password**.
This shows **how SQL injection happens**.

Parameterized Query

A **parameterized query** (also called a **prepared statement**) is a secure way of writing SQL commands where **user input is never mixed directly into the SQL string**.
Instead, you create the SQL query with **placeholders**, and then pass the user values **separately**.
Because the database treats these values as **data only**, attackers cannot turn them into SQL code.

Unsafe Query (Vulnerable to SQL Injection)

query = "SELECT * FROM users WHERE username='" + user + "';"
Here the user input becomes part of the SQL command.
If an attacker enters: aashu' OR '1'='1 the query becomes dangerous.

Safe Query Using Parameterization

query = "SELECT * FROM users WHERE username=?"
cur.execute(query, (user,))

Why This Prevents SQL Injection

- The query is **parsed first**, locking its structure.
- User input is **inserted later**, treated strictly as text.
- Even if attacker enters malicious input, the database sees only **literal values**, not commands.

2. How to Prevent SQL Injection

Safe Example (Using Prepared Statements)

```
import sqlite3
conn = sqlite3.connect(":memory:")
cur = conn.cursor()
# create table
cur.execute("CREATE TABLE users(username TEXT, password TEXT)")
cur.execute("INSERT INTO users VALUES('aashu', '1234')")
# user input
username = input("Enter username: ")
password = input("Enter password: ")
# SAFE: using parameterized query
query = "SELECT * FROM users WHERE username=? AND password=?"
cur.execute(query, (username, password))
result = cur.fetchall()
if result:
   print("Login successful")
else:
   print("Login failed")
```

Explanation

- ? acts as **placeholders** for values.
- The database treats user values as **data**, not as **SQL code**.
- Even if the attacker types ' OR '1'='1, it is treated as a *string*, not as SQL.
- This completely blocks SQL injection.

3. Cross-Site Request Forgery (CSRF)

CSRF is an attack where a hacker **forces a logged-in user's browser to perform actions without their permission**.

Why it works

- Browsers automatically send **cookies** (login tokens) to a website whenever a request is made.
- If you are logged into a website (like your bank), your browser will send your bank cookie with every request to that site.
- A hacker can make a hidden request to that same site using your browser.

Simple Scenario

1. You log into **mybank.com**
   → Your browser gets a cookie: session=abcd1234.
2. Now you open **malicioussite.com** (maybe from a link, video site, or ad).
3. That website secretly contains a hidden form:

```
<form action="https://mybank.com/transfer" method="POST">
  <input type="hidden" name="amount" value="50000">
  <input type="hidden" name="toAccount" value="12345">
</form>
<script>document.forms[0].submit();</script>
```

4. Your browser **automatically sends your bank cookie** with this request.
5. So the bank receives:
   - amount = 50000
   - toAccount = 12345
   - with a **valid login cookie**

The bank thinks **you** sent the request because the cookie belongs to you.

Result

Money transferred → without your knowledge.

Real-life example

Imagine someone forging your signature on a cheque.
Your browser is like your "signature" and the attacker makes you sign something you didn't mean to.

4. Open Redirect Attack

Open Redirect happens when a website redirects users to a location that comes from **user input**, without checking if it's safe or not.

Why websites use redirect URLs

If you try to visit a page but you're not logged in:

- Website sends you to /login?returnUrl=/cart
- After login, it redirects you to /cart.

This is normal behavior.

How attackers exploit it

Attackers replace returnUrl with their own malicious link.
Example:
https://legitwebsite.com/login?returnUrl=http://phishing-site.com

You think you're logging into a trusted website, but after login:

- The real site redirects you to the attacker's link.
- The malicious site then steals your password or details.

What attacker gains

- Users think the redirect is legitimate because it started from the real website.
- They trust the next page and enter personal information.

Real-life example

Imagine a trusted restaurant telling you:
“After checking in at the counter, go to Room A.”
An attacker replaces the sign:
“After checking in, go to Room Z.”
You still trust it because you came through the main entrance.

How to Prevent Open Redirects

1. Use LocalRedirect()

ASP.NET Core checks if the redirect URL belongs to **your own website**.
If not, it throws:

The supplied URL is not local.

2. Validate Using Url.IsLocalUrl()

```
if (Url.IsLocalUrl(returnUrl))
   return Redirect(returnUrl);
else
   return Redirect("/home");
```

Only redirects inside your site are allowed.

Web Servers

A **web server** is a program or process that **hosts web applications**. It listens for **HTTP requests** (like someone opening your website) and sends back **content and services** (HTML pages, images, JSON data, etc.).

- Web servers work through **TCP ports**, usually **port 80 (HTTP)** or **port 443 (HTTPS)**.
- They allow web applications to **process incoming messages** and respond to users.

Some common web servers to host **ASP.NET Core apps**:

- Microsoft **IIS**
- **Apache**
- **NGINX**

1. IIS Web Server

IIS (Internet Information Services) is developed by **Microsoft**.

- Runs **only on Windows**.
- Comes pre-installed with **Windows Server** editions (so not exactly free).
- Works well if you are already familiar with the **Windows ecosystem**.

Advantages of IIS

Advantage	Explanation
Microsoft support	Official support and regular updates from Microsoft
.NET framework support	Can run ASPX scripts and integrate seamlessly with .NET apps
Integration with Microsoft services	Easy connection with MS SQL, ASP, and other Microsoft tools

2. Apache Web Server

Apache is an **open-source web server** developed by the **Apache Software Foundation**.

- Runs on **almost every operating system** (Windows, Linux, macOS), but most commonly with **Linux**.
- Focused on being **robust, secure, and HTTP-compliant**.

Advantages of Apache

Advantage	Explanation
Open source	Free to use, no licensing fees
Flexible	Modules allow customization for different use cases
Secure	High level of security with regular updates
Strong community	Active support from a large user base

3. NGINX

NGINX is a **high-performance, lightweight web server** developed by **Igor Sysoev**.

- Can act as a **HTTP server, reverse proxy, or mail proxy**.
- Efficiently handles **huge traffic** with **low hardware resources**.

Advantages of NGINX

Advantage	Explanation
Open source	Free to use with a strong developer community
High-speed	Lightweight, fast, and scalable
Reverse proxy	Can forward requests to other servers or handle load balancing
Works well on VPS	Ideal for **virtual private servers** and cloud hosting

Hosting Models in ASP.NET Core

Hosting is the process of **running an ASP.NET Core application on a web server** so that it can respond to user requests over the internet or a network. It determines **how and where the app will run** (e.g., Kestrel, IIS, NGINX).

Deploying is the process of **moving your ASP.NET Core application from development to a server** so that users can access it. It involves **publishing the app, copying files to a server, and configuring the server** to run the application.

When you want to **host and deploy an ASP.NET Core application**, there are **two main hosting models**:

1. Out-of-Process Hosting Model

In this model, the app runs on the **Kestrel server internally**, and optionally, a **proxy server** like IIS, NGINX, or Apache can forward requests to it.

- **Kestrel Server:**
  - Default web server in ASP.NET Core
  - Cross-platform and lightweight
  - Can serve requests directly
- **Proxy Server:**
  - Handles advanced tasks like HTTPS, load balancing, and serving static files
  - Forwards requests to Kestrel (reverse proxy)
  - Both servers run at the same time

Example:
A small API runs on Kestrel directly on Linux VPS. For a bigger app, IIS acts as a proxy to Kestrel, forwarding all user requests.

2. In-Process Hosting Model

- Only **one server** is used, like IIS or NGINX.
- The app runs **directly inside the server**.
- Kestrel is **not directly exposed**, and the server uses its own HTTP implementation to host the app.

Example:
A company portal is hosted directly inside IIS. The app runs inside IIS without Kestrel being involved, which improves performance.

Steps to Deploy ASP.NET Core to IIS

1. **Publish to Folder**
   - In Visual Studio, choose **Publish → Folder** to generate the app files.
2. **Copy Files to IIS Location**
   - Move the published files to a folder on the IIS server, e.g., C:\inetpub\wwwroot\MyApp.
3. **Create Application in IIS**
   - Open IIS Manager → Right-click **Sites** → Add Application → Point to the folder.
4. **Load the App**
   - Open the browser and navigate to your app URL to make sure it works.

ASP.NET Core Module (ANCM)

The **ASP.NET Core Module** is an IIS module that connects IIS with ASP.NET Core apps.
It works in **two ways**:

1. **In-process hosting**
   - App runs **inside IIS worker process (w3wp.exe)**
   - Uses **IIS HTTP Server**
   - Kestrel is not exposed
2. **Out-of-process hosting**
   - App runs on **Kestrel internally**
   - IIS **forwards all requests** to Kestrel
   - Works **only with Kestrel**, not HTTP.sys

Example:
- Out-of-process: IIS forwards requests to Kestrel where the app runs internally.
- In-process: IIS directly runs the app using its own HTTP server without Kestrel.

Docker and Containerization

What is Docker?

Docker is a **platform** that helps you **build, package, and deploy applications** in a consistent way. It uses **operating system-level virtualization**, which means it can run applications in isolated environments without needing a full virtual machine.

- To run a Docker application, you need:
  1. **Docker Image**
  2. **Docker Container**

What is a Docker Image?

A **Docker image** is like a **blueprint or recipe** for your application. It contains:

- All configuration and instructions needed to create a container
- Dependencies, libraries, and app code

Key Points:
- Created when building the application using a **Dockerfile**
- **Read-only**: Once created, you cannot modify it directly
- Can be deleted from Docker if not needed

Example:

Think of a Docker image as a **pre-packaged app installer**—you can use it to create as many containers as you want, but the image itself doesn't change.

What is a Docker Container?

A **container** is a **running instance of a Docker image**. It is like a **sandboxed environment** that has:

- Its own networking
- Its own file system
- Isolated process execution

Key Points:

- Created based on the Docker image
- Runs on the **Docker Engine**
- Can have multiple containers from the same image

Example:

If the Docker image is a cake recipe, the container is the **actual cake baked from that recipe**, ready to eat.

Difference between Virtual Machine and Docker Container

Feature	Virtual Machine (VM)	Docker Container
Operating System	Full OS installed; shares host lifecycle	Uses host OS; lightweight, only runs necessary components
Function	Acts like a full PC (RAM, Hard disk, networking, etc.)	Minimal, isolated environment to run app
Speed	Slower due to full OS	Faster, lightweight
Engine	Runs on host system	Runs on Docker Engine; no full OS needed

Azure Cloud

How to Publish ASP.NET Core Web App in Azure

1. **Publish Option**
   - Right-click your project in Visual Studio → **Publish**
2. **Select Azure**
   - Choose **Azure** as the target platform
3. **Select Azure App Service**
   - Click **Next** → In the right pane, choose **Azure App Services (Windows)**
4. **Create App Service Instance**
   - Click the **(+) sign** → Fill in **Name**, **Subscription**, **Hosting Plan**
   - For **Hosting Plan**, click **new**, choose **Location** and **Size** → Select **Free** → Click **Ok** → Click **Create**
5. **Select the Instance**
   - After creation, select the newly created **App Service Instance** → Click **Next**
6. **Skip API Management**
   - Optional: skip API management by checking the box

After this, your **ASP.NET Core app is published to Azure** and accessible online.